Version Control System

Overview

This project uses Git as the version control system, hosted on GitHub at https://github.com/pamuditha-senanayake/LMS_IUAP.git

GitHub Workflow

The project implements GitHub Actions for continuous integration (CI). From .github/workflows/ci.yml, the workflow:

- Triggers on push and pull requests to the main branch
- Runs automated tests on Ubuntu
- Uses Maven for backend Java build
- Uses npm for frontend build

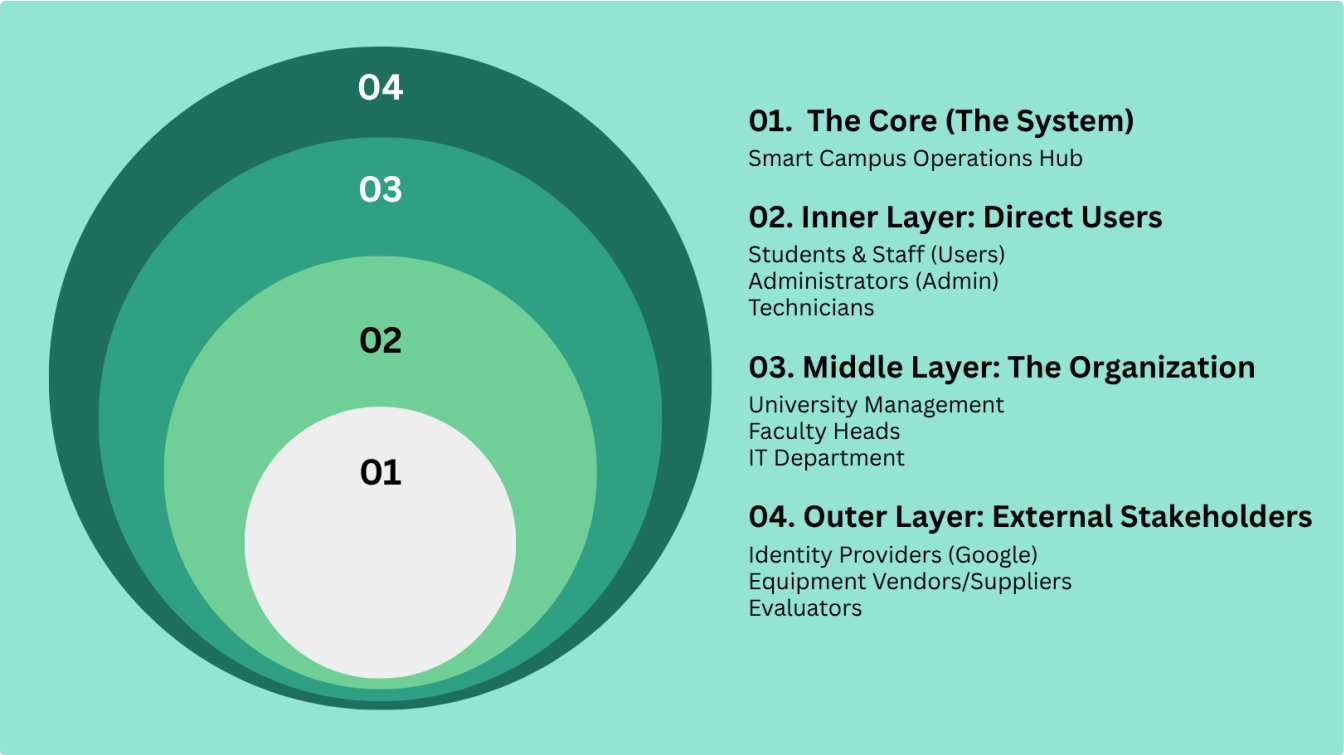
Repository Configuration

Remote: origin → https://github.com/pamuditha-senanayake/LMS_IUAP.git

Default Branch: main

Workflow File: .github/workflows/ci.yml

Onion Diagram (Stakeholder Analysis)



System Overview

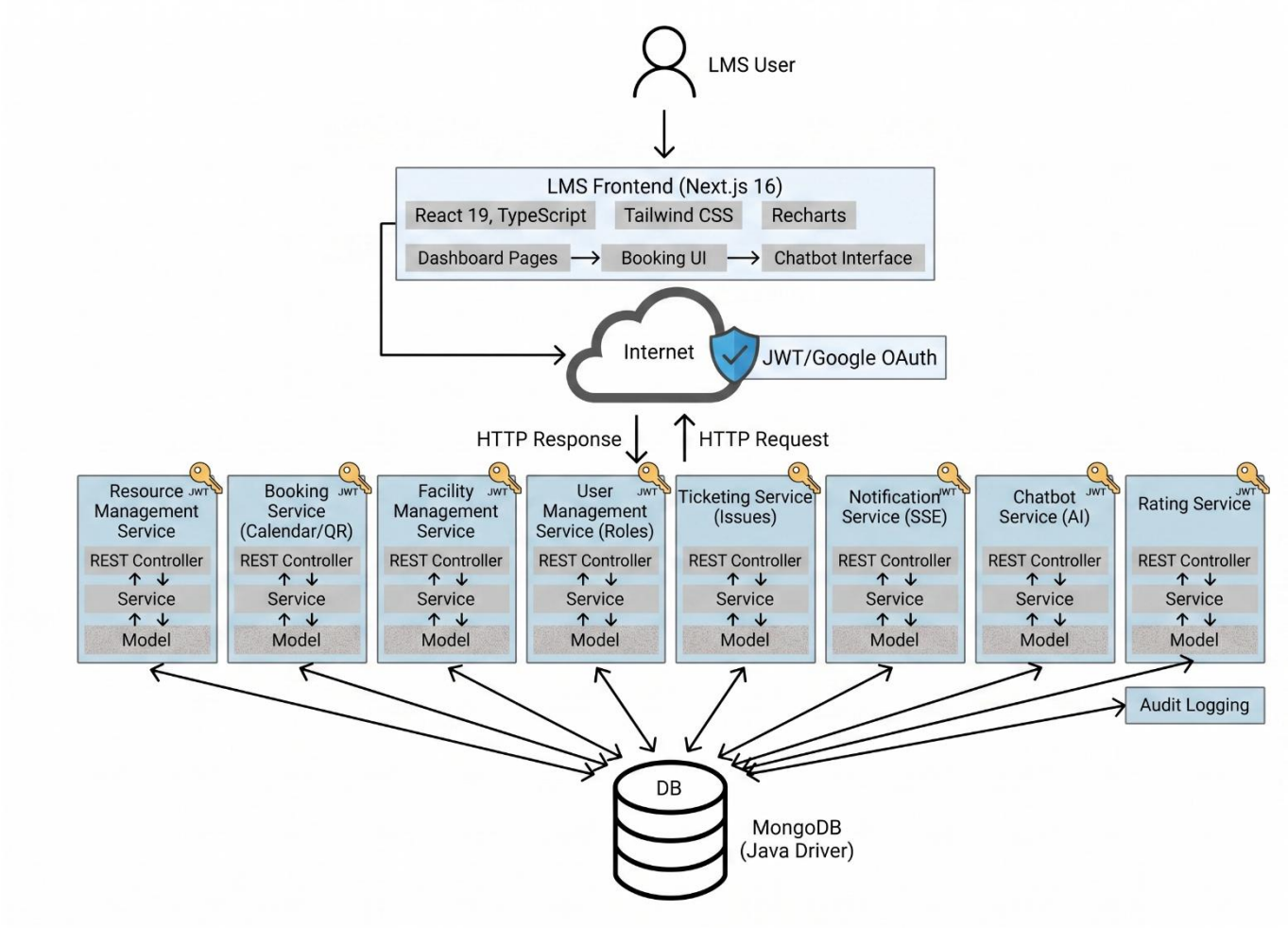
LMS (Learning Management System)

This is a full-stack web application for managing university/learning center resources, bookings, and facilities.

Technology Stack

Layer	Technology
Frontend	Next.js 16 (React 19, TypeScript), Tailwind CSS, Recharts
Backend	Spring Boot 3 (Java 17), MongoDB
Authentication	JWT (JSON Web Tokens), Google OAuth
Testing	Vitest (frontend), JUnit (backend)
CI/CD	GitHub Actions

System Architecture



Core Features

1. Resource Management - Book/manage learning resources (labs, equipment)
2. Booking System - Schedule resources with calendar view, QR codes
3. Facility Management - View/edit facility details
4. User Management - Admin, Lecturer, Student roles
5. Ticketing System - Report issues, track status
6. Notifications - Real-time alerts for bookings
7. Smart Chatbot - AI-powered assistance for bookings
8. Rating System - Rate facilities/resources
9. Audit Logging - Track all system actions

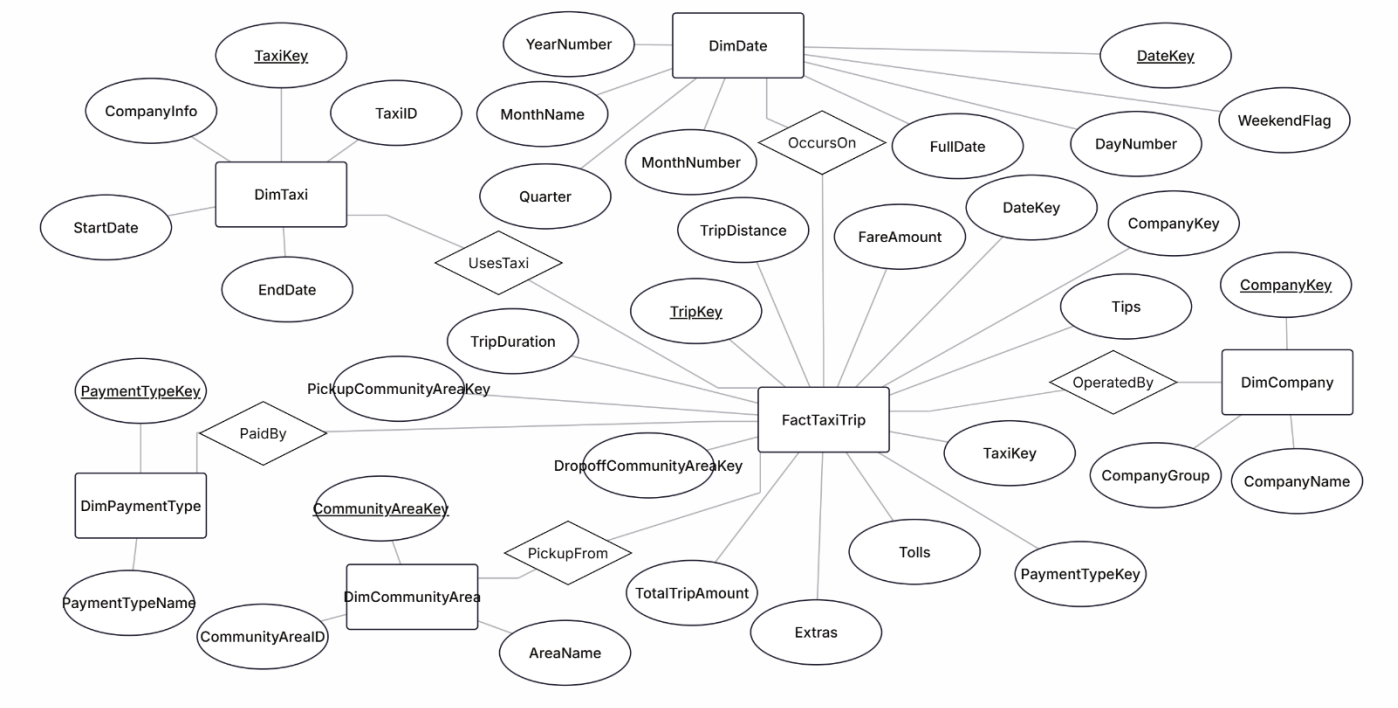
Key Modules

1. **BookingService** - CRUD for bookings, availability checks
2. **FacilitiesService** - Facility management
3. **UserService** - User registration/authentication
4. **TicketingService** - Issue tracking
5. **NotificationService** - Real-time SSE notifications
6. **ChatbotService** - AI chatbot integration
7. **RatingService** - Facility ratings

Security

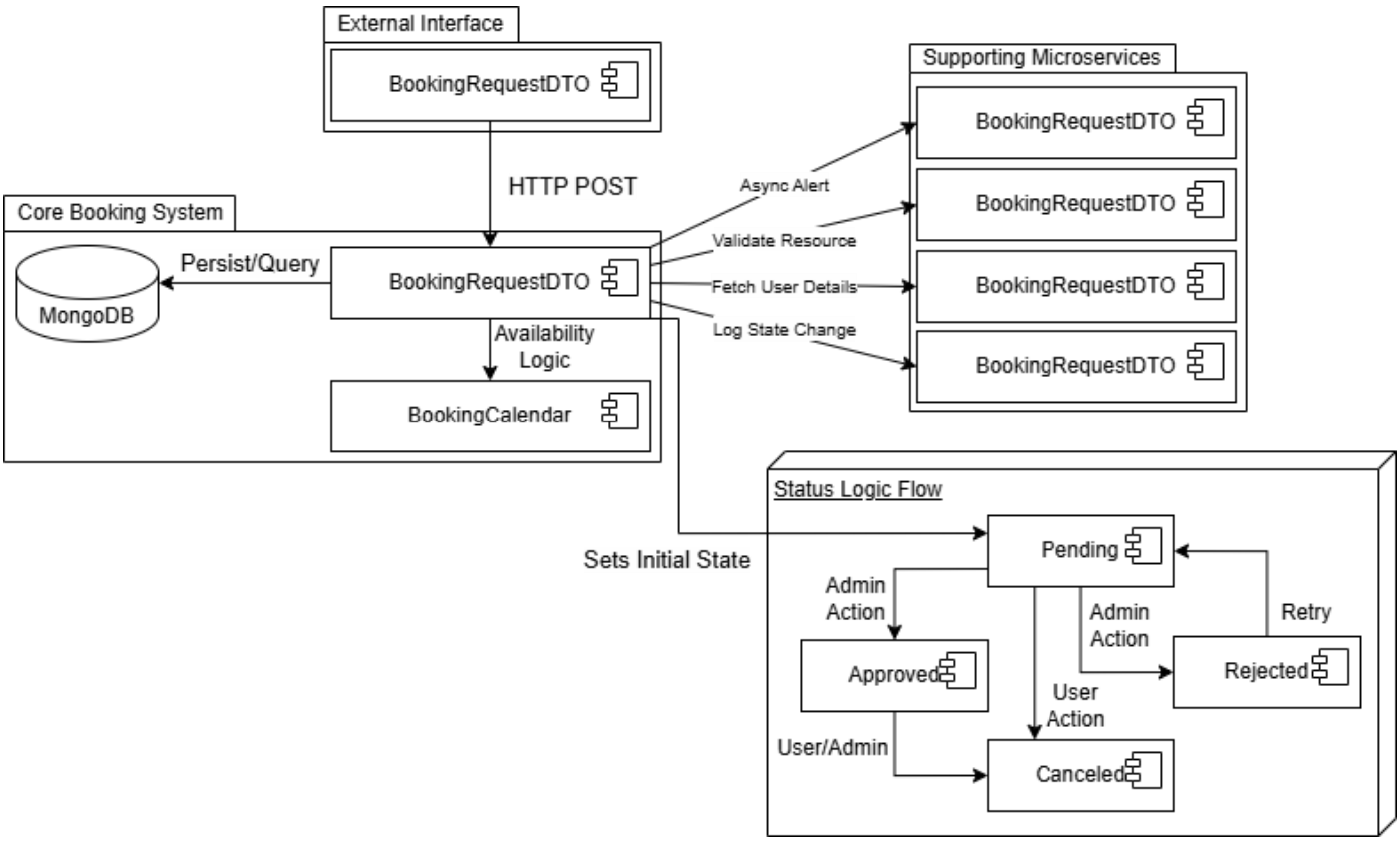
1. JWT-based authentication
2. Role-based access control (Admin, Lecturer, Student)
3. Password encryption (BCrypt)
4. CORS configuration

ER Diagram



IT23142428 Weerakoon W.M.I.U.
Booking workflow + conflict checking

Service Design



CRUD Operations: Booking Workflow & Conflict Checking

Get All Bookings
URL - http://localhost:8080/api/bookings?page=0&size=10

Request Type – GET

Create Booking
URL - http://localhost:8080/api/bookings

Request Type – POST

Update Booking
URL - http://localhost:8080/api/bookings/{{bookingId}}

Request Type – PUT

Update Booking Status (Admin)
URL

http://localhost:8080/api/bookings/{{bookingId}}/status?status=APPROVED&adminId={{userId}}&reason=Approved

Request Type – PATCH

Delete Booking (Admin)
URL – http://localhost:8080/api/bookings/{{bookingId}}

Request Type – DELETE

Backend Operations

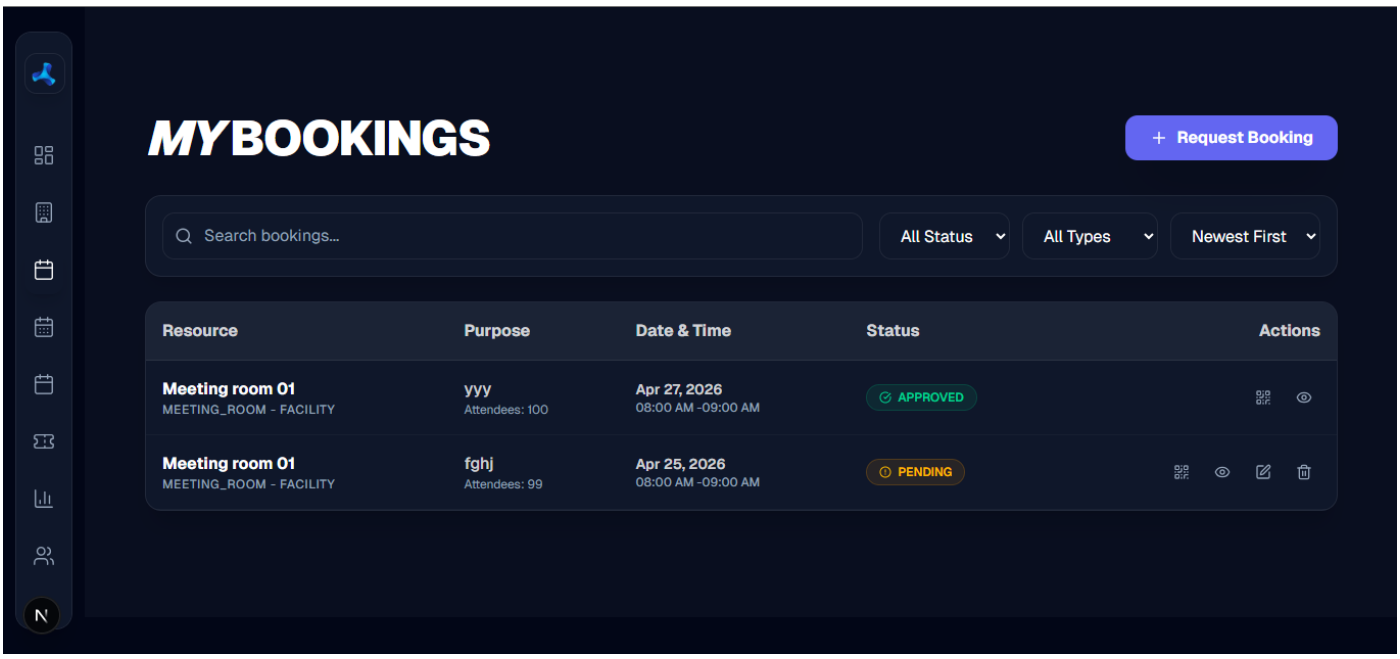
- Creating new booking requests in the bookings collection, including resource ID, purpose, expected attendees count, start/end time, type, and user summary (ID, name, email).
- Validating booking requests, ensuring resourceId and purpose are provided, start time is before end time, attendees count is at least 1, and booking time is at least 15 minutes in the future.
- Performing conflict checking by querying findConflictingBookings() to detect overlapping time slots for the same resource, rejecting with 409 CONFLICT if a booking already exists during the requested period.
- Checking resource availability before booking creation, verifying the resource is not marked as OUT_OF_SERVICE or under maintenance.
- Retrieving all bookings from the database with pagination support, filtering by userId, resourceId, status, and date range.
- Fetching booking details by ID, including full resource information, requester/reviewer details, status history, and timestamps.
- Updating booking status (admin-only) with valid transition rules: PENDING → APPROVED/REJECTED, APPROVED → CANCELLED, REJECTED → PENDING for retry.
- Approving bookings with additional conflict re-check to ensure no overlapping approved bookings exist for the resource during that time.
- Recording status history in booking_status_history collection, tracking old status, new status, changed by user ID, note/reason, and timestamp.
- Creating notifications to inform users when their booking is approved, rejected, or cancelled.
- Allowing users to cancel only their own bookings that are PENDING or APPROVED, recording the cancellation in history.
- Deleting bookings (admin-only) to permanently remove from database after verification.

Frontend Operations

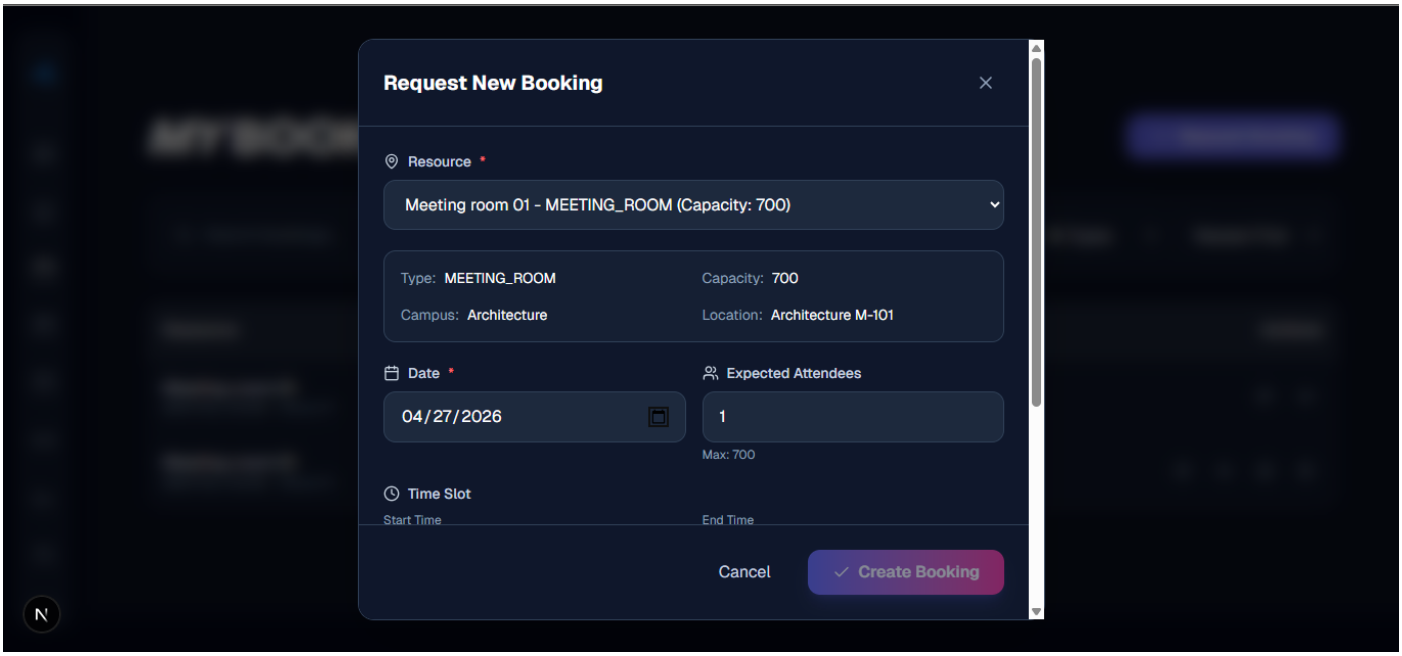
- Providing a booking request form where users can select a resource, enter purpose, set expected attendees, and choose start/end time using a date-time picker.
- Displaying real-time availability by showing hourly time slots (8 AM – 8 PM) with color-coded indicators for available and booked slots.
- Validating inputs before submission: checking that start time is in the future, end time is after start time, and attendee count is valid.
- Showing conflict warnings immediately if the selected time slot conflicts with an existing booking, displaying the conflicting booking details.
- Displaying a booking management dashboard for users showing their pending, approved, rejected, and cancelled bookings.
- Allowing users to view the full status history of each booking, including who approved/rejected and any notes provided.
- Providing admin booking overview with filters by status, resource, date range, and paginated results sorted by latest activity.
- Showing booking statistics for admins: total bookings by status, breakdown by resource type, and monthly trends.
- Allowing admins to approve/reject bookings with an optional reason field, triggering notifications to the requesting user.

- Enabling users to cancel their own bookings from the dashboard with a confirmation dialog.
- Displaying resource details alongside each booking including name, type, location (campus, building, room number), and capacity.

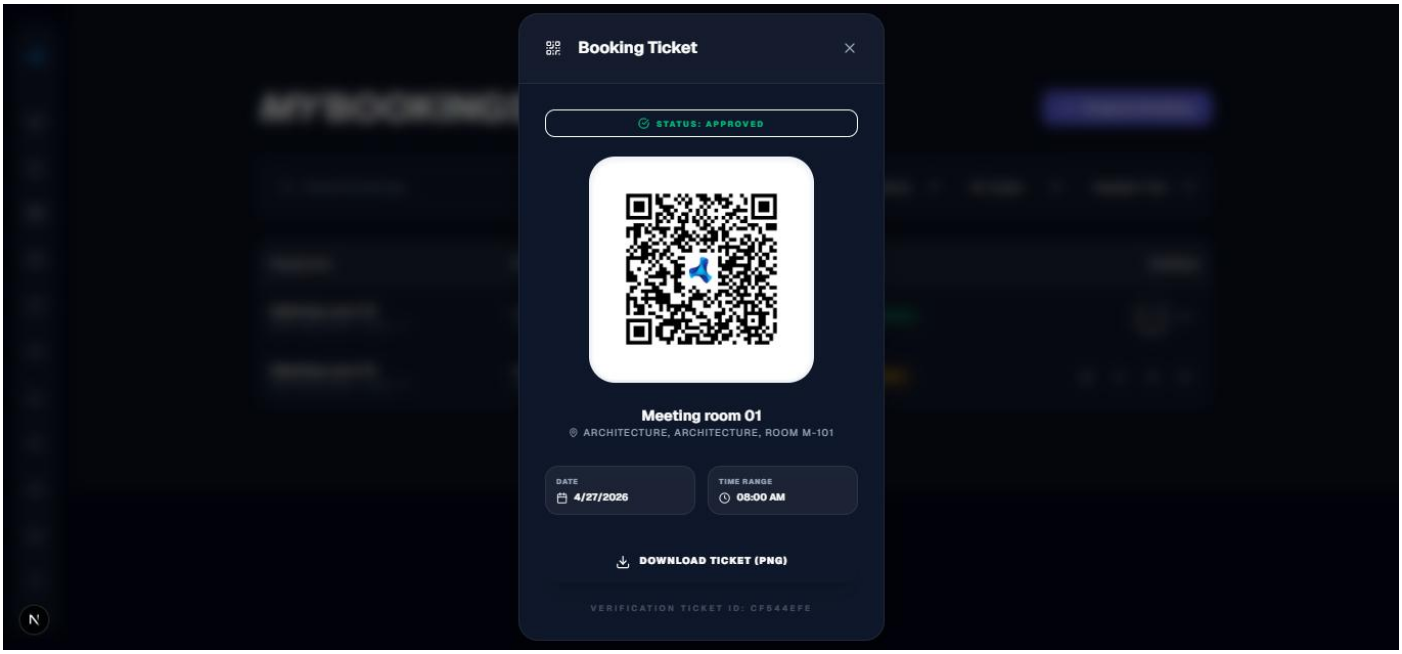
My Booking Page



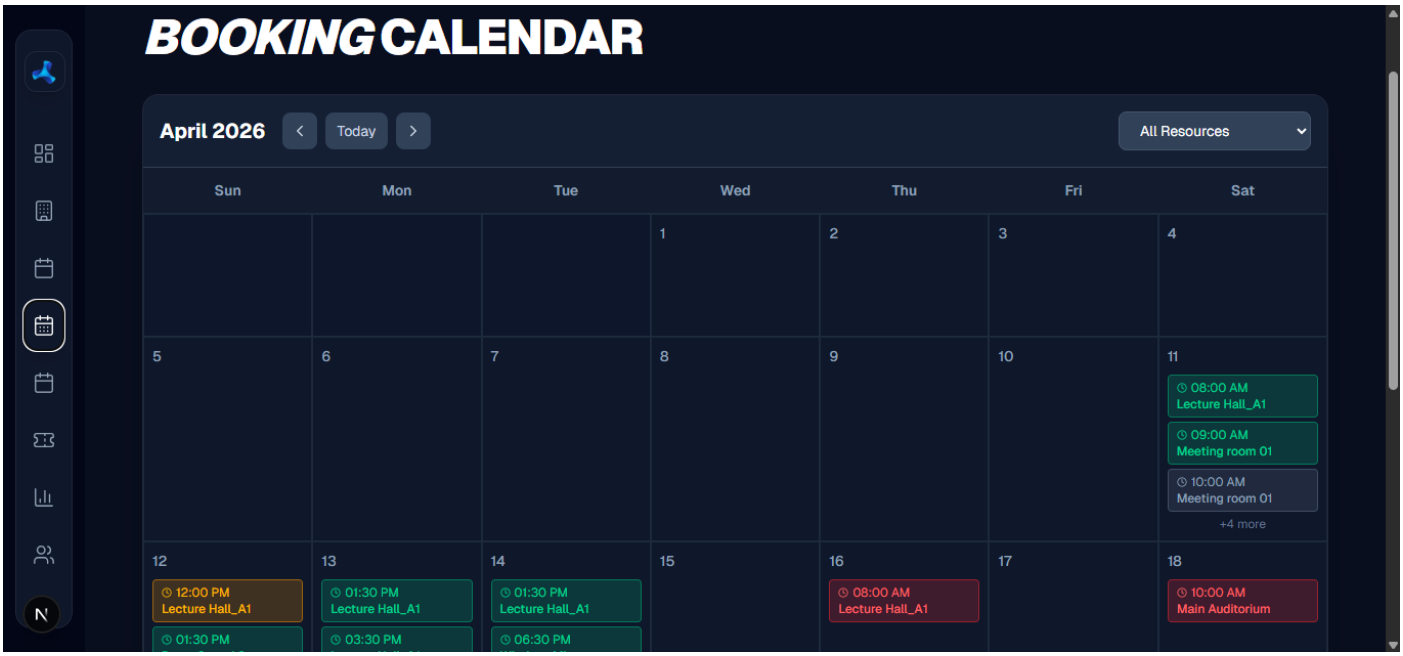
Request New Booking Form



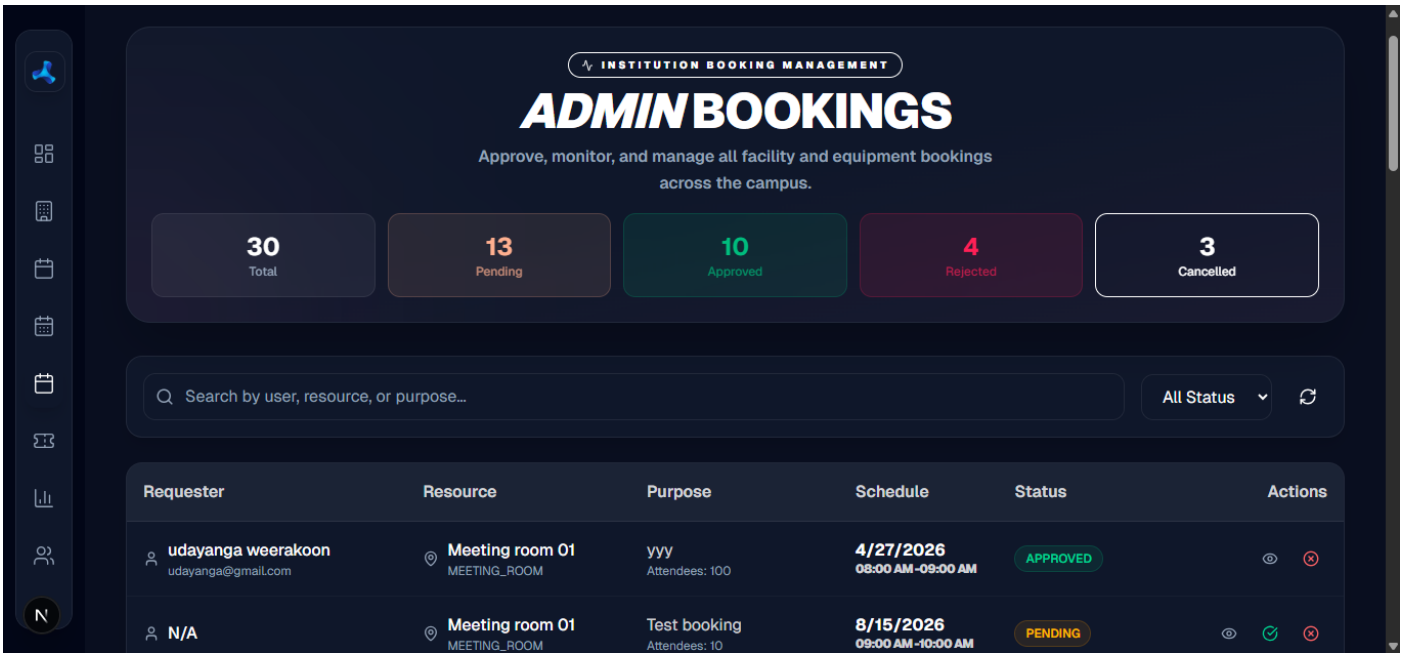
QR Generator



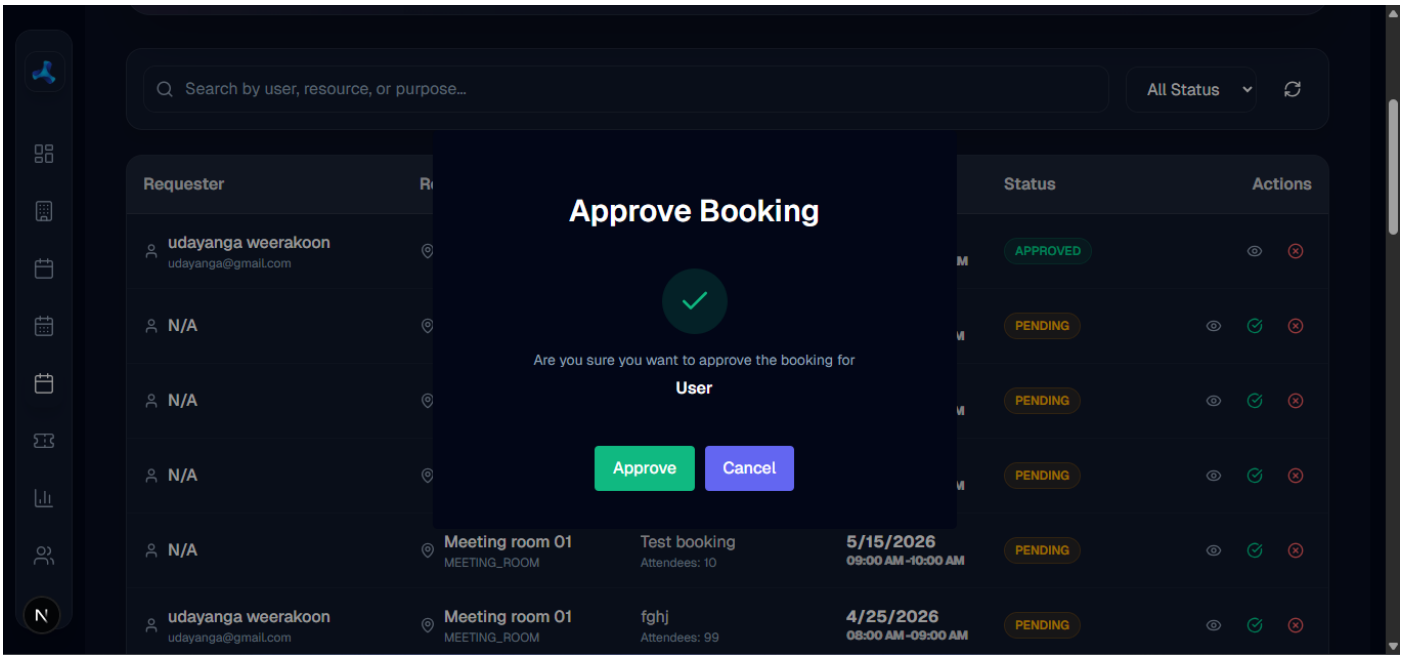
Booking Calendar



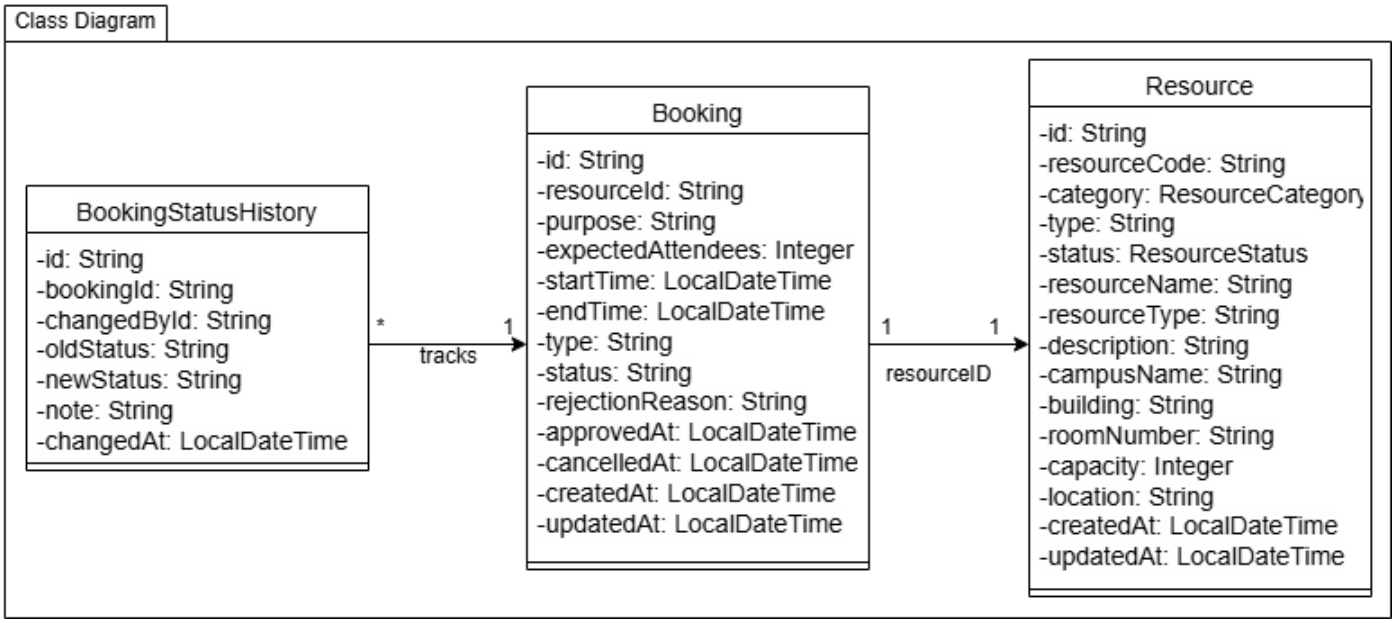
Booking Management Page



Approve/Reject Booking

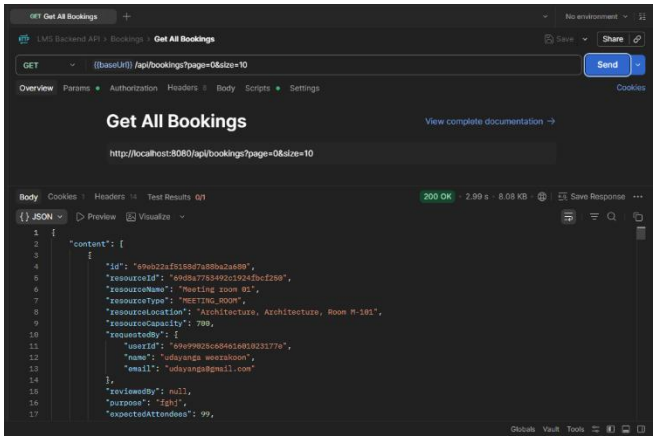


Class Diagram

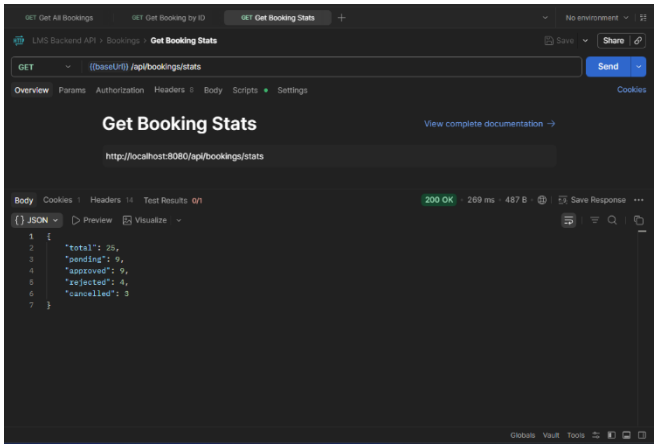


Testings

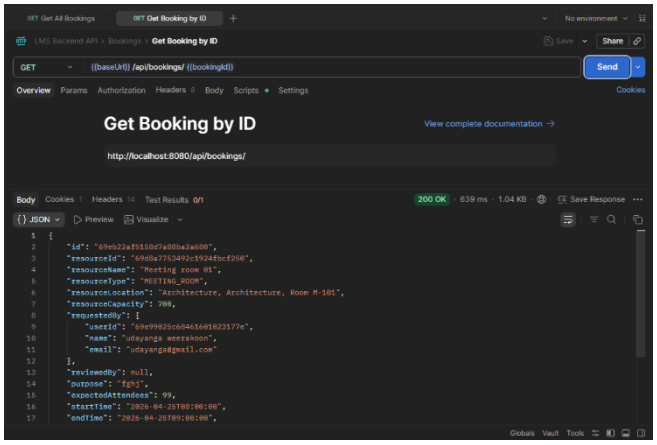
Get all Bookings



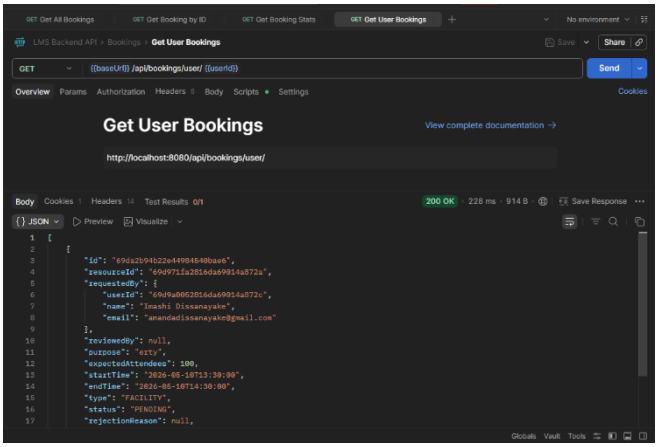
Get Booking Stats



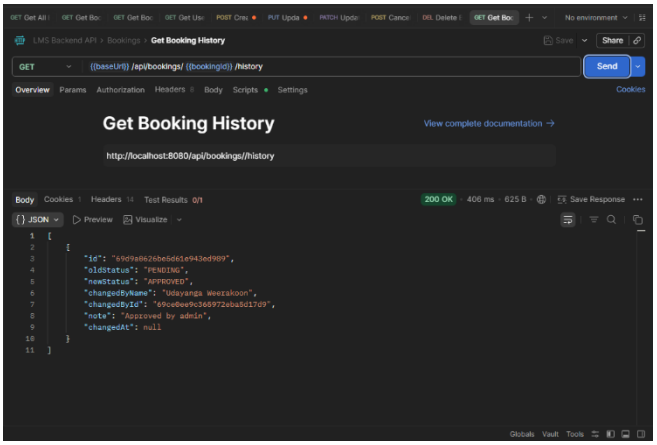
Get Booking by ID



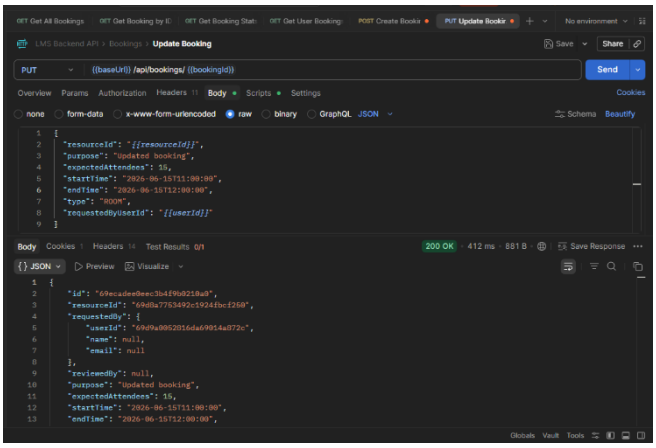
Get User Bookings



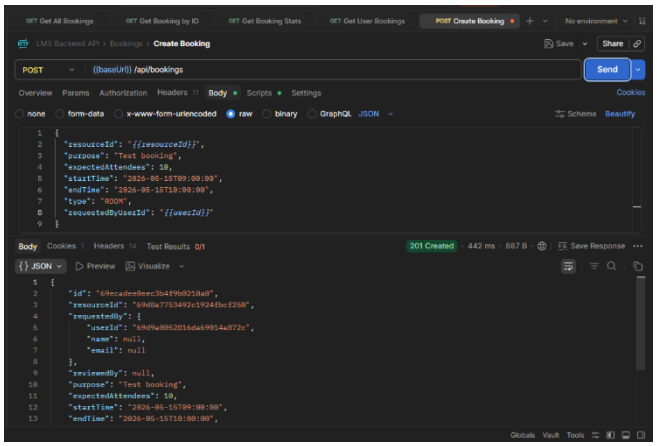
Get Booking History



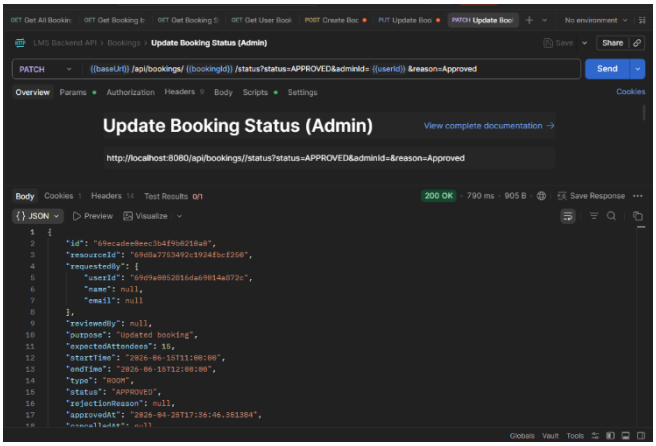
Update Booking



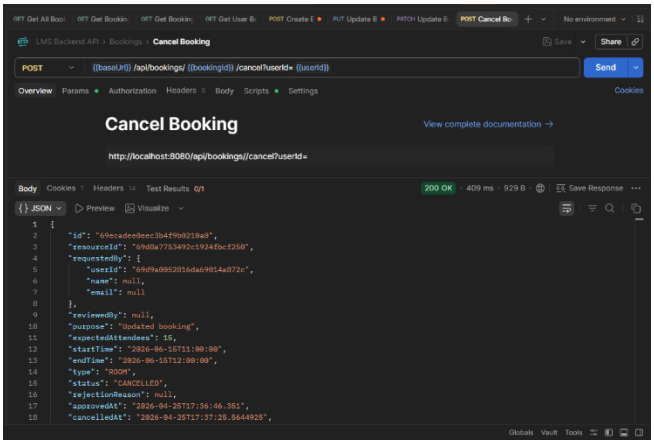
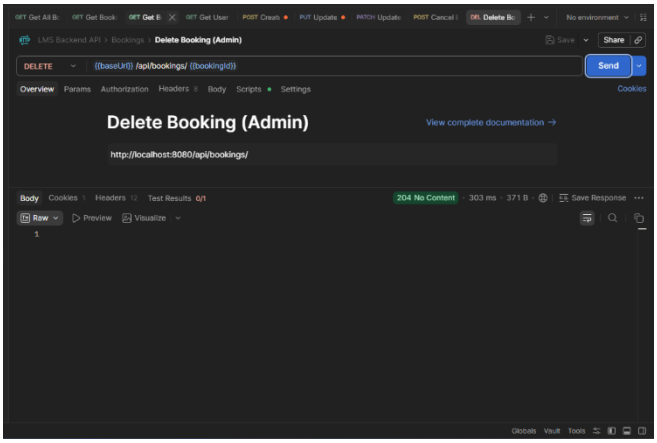
Create Booking



Update Booking Status (Admin)



Cancel Booking



Delete Booking (Admin)